Breaking Down OOP in Bus Booking Backend

Encapsulation

Basically, encapsulation is just making sure an object's internal stuff stays hidden and people only mess with it through a proper interface.

What's already there

1.

In our Busly.API/Models, we use C# properties instead of just public fields. Like in the Customer model, we keep the PasswordHash tucked away properly.

```
// Busly.API/Models/Customer.cs
public class Customer
{
   public Guid Id { get; set; }
   public string PasswordHash { get; set; } = null!; // never plain text
   public string Email { get; set; } = null!;
   public bool TcAccepted { get; set; }
}
```

2.

The EmailService hides the actual communication channel so nothing from the outside can mess with the message queue.

```
public class EmailService : IEmailService
{
   private readonly Channel<EmailMessage> _channel; // private — nobody writes to this directly

    public ChannelReader<EmailMessage> Reader => _channel.Reader; // read-only for consumers

   public async Task EnqueueBookingConfirmationAsync(Booking booking, byte[]? pdfBytes = null)
```

```
    {
        // Only this class can write to the channel
        await _channel.Writer.WriteAsync(message);
    }
}
```

3.

Our SeatLockCleanupService handles its own timer and cleanup logic without the rest of the app needing to know how it works.

Better Approach: Better Booking Statuses

Right now, we use strings for booking statuses, which is a bit messy. It would be way cleaner to use an Enum and methods to handle the changes.

```
// Busly.API/Models/Booking.cs
public class Booking
{
    private BookingStatus _status = BookingStatus.Initiated;

    public string Status
    {
        get => _status.ToString().ToUpper();
    }

    public void Confirm()
    {
        if (_status != BookingStatus.PaymentPending)
            throw new InvalidOperationException("Invalid transition.");
        _status = BookingStatus.Confirmed;
    }
```

Inheritance

Inheritance lets us build hierarchies so we don't have to write the same code over and over again.

What's already there

1. Using Framework Base Classes

We use .NET's BackgroundService for our cleanup tasks and ControllerBase for our API controllers, which saves us a ton of time.

```
// Busly.API/Services/SeatLockCleanupService.cs
public class SeatLockCleanupService : BackgroundService
{
   protected override async Task ExecuteAsync(CancellationToken stoppingToken)
   {
       while (!stoppingToken.IsCancellationRequested)
       {
           await CleanupExpiredLocksAsync();
           await Task.Delay(_cleanupInterval, stoppingToken);
       }
   }
}
```

2. Standard Interfaces

Jobs like EmailDispatchJob use IHostedService so they always start and stop the same way.

```
/ EmailDispatchJob — now just the unique logic
public class EmailDispatchJob : BaseHostedJob
{
   protected override async Task RunAsync(CancellationToken ct)
   {
       await foreach (var message in _emailService.Reader.ReadAllAsync(ct))
           await ProcessMessageAsync(message, ct);
   }
}
```

Better Approach: A Shared Job Base

We could make a BaseHostedJob to get rid of all that repetitive code in our background jobs.

```
// Suggested: Busly.API/BackgroundJobs/BaseHostedJob.cs
```

```
public abstract class BaseHostedJob : IHostedService
{
   private Task? _executingTask;
   private CancellationTokenSource? _cts;

   public Task StartAsync(CancellationToken cancellationToken)
   {
       _cts = CancellationTokenSource.CreateLinkedTokenSource(cancellationToken);
       _executingTask = RunAsync(_cts.Token);
       return Task.CompletedTask;
   }
```

Polymorphism

Polymorphism means we can treat different objects as the same type, making our code super flexible.

What's already there

1.

We use interfaces like IBookingService so our controllers don't care about the actual implementation

```
// Busly.API/Controllers/BookingController.cs
public class BookingController : ControllerBase
{
   private readonly IBookingService _bookingService; // interface, not BookingService

   public BookingController(IBookingService bookingService)
   {
           _bookingService = bookingService; // could be BookingService, a mock, or a
future v2
   }
}
```

2.

By overriding ExecuteAsync in our cleanup service, we can plug our custom logic right into the framework.

```
// Busly.API/Services/SeatLockCleanupService.cs
public class SeatLockCleanupService : BackgroundService
{
   protected override async Task ExecuteAsync(CancellationToken stoppingToken) { ... }
}
```

Better Approach: One Interface for All Users

Instead of long if/else blocks for logins, we should use an IAppUser interface to handle Customers and Admins the same way.

```
// Suggested: Busly.API/Models/IAppUser.cs
public interface IAppUser
{
   Guid Id { get; }
   string Email { get; }
   string PasswordHash { get; }
   string Role { get; }
}

// Customer.cs — just add the interface
public class Customer : IAppUser
{
   public string Role => "Customer";
   // ... existing properties stay the same
}

// BusOperator.cs
public class BusOperator : IAppUser
{
   public string Role => "Operator";
}

// Admin.cs
public class Admin : IAppUser
```

```
{
    public string Role => "Admin";
}
```

Abstraction

Abstraction is all about focusing on "what" something does rather than "how" it does it.

What's already there

Interface	What it Hides
IBookingService	Prices, coupons, and PDF creation logic.
ISeatRepository	Messy SQL queries and data stuff.
IEmailService	The actual background queuing for emails.
IConfigService	Where settings come from (DB or JSON).

Better Approach: A User Model Hierarchy

We should use an abstract AppUser base class for all our user types to centralize common fields like Email.

```
// Suggested: Busly.API/Models/AppUser.cs
public abstract class AppUser
{
    public Guid Id { get; set; }
    public string Email { get; set; } = null!;
    public string PasswordHash { get; set; } = null!;
    public DateTime? CreatedAt { get; set; }

    public abstract string DisplayName { get; } // each type defines its own
    public abstract string Role { get; }
}

// Customer.cs
```

```
public class Customer : AppUser
{
   public override string DisplayName => Name ?? Username;
   public override string Role => "Customer";
   // ... Customer-specific fields
}

// BusOperator.cs
public class BusOperator : AppUser
{
   public override string DisplayName => CompanyName;
   public override string Role => "Operator";
   // ... Operator-specific fields
}

// Admin.cs
public class Admin : AppUser
{
   public override string DisplayName => Username;
   public override string Role => "Admin";
}
```